

Avance de Proyecto: PokeHash

Integrantes:

Cristóbal Sazo

Sebastián Riveros

Simón Guzmán

Bruno Orellana

Sección 1: Descripción de la Aplicación Computacional Propuesta

1.1 Propósito y Visión General

PokeHash es un gestor táctico e interactivo de consola diseñado para administrar información del universo Pokémon. La aplicación resuelve la necesidad de los jugadores de planificar y evaluar matemáticamente sus equipos antes de los combates, permitiéndoles consultar rápidamente estadísticas y debilidades sin depender de calculadoras o wikis externas en internet.

El público objetivo abarca desde fanáticos casuales de la franquicia que desean explorar el catálogo, hasta jugadores competitivos (*VGC* o *Smogon*) que requieren una herramienta rápida y precisa para la toma de decisiones estratégicas.

1.2 Lista Detallada de Funcionalidades

1. **Cargar Catálogo:** Lectura e inicialización masiva de la base de datos de Pokémon desde un archivo .csv al iniciar la aplicación.
2. **Buscar Pokémon:** Consulta directa de la ficha técnica ingresando el nombre exacto o su número correspondiente en la Pokédex.
3. **Filtrar por Tipo:** Despliegue de una lista con todos los Pokémon que comparten un tipo elemental (ej. "Agua").
4. **Agregar al Equipo:** Selección e incorporación de un Pokémon al equipo activo (con un límite estricto de 6 integrantes).
5. **Ver Equipo Actual:** Visualización de los miembros seleccionados y sus estadísticas.
6. **Analizar Debilidades:** Generación de un reporte automático que cruza los tipos del equipo y muestra las vulnerabilidades conjuntas.
7. **Deshacer Acción (Undo):** Eliminación rápida del último Pokémon agregado al equipo en caso de error táctico.
8. **Exportar Equipo:** Guardado del equipo actual y su análisis en un archivo de texto local.

1.3 Mapa de Navegación (Menús)

Menú Principal

1. Explorar Pokédex
 - 1.1. Buscar por Nombre
 - 1.2. Buscar por Número de Pokédex
 - 1.3. Filtrar por Tipo
2. Gestión Estratégica de Equipo
 - 2.1. Agregar Pokémon al Equipo
 - 2.2. Ver Equipo Actual
 - 2.3. Analizar Debilidades del Equipo
 - 2.4. Deshacer última adición
3. Exportar Equipo a Archivo
4. Salir

1.4 Flujo de Interacción (La App como "Caja Negra")

Funcionalidad: Agregar Pokémon al Equipo y Deshacer

- **Paso 1:** En el menú principal, el usuario presiona "2" para entrar a Gestión de Equipo y luego "1" para Agregar Pokémon.
- **Paso 2:** La consola muestra el mensaje: *"Ingrese el nombre del Pokémon a reclutar:"*.
- **Paso 3:** El usuario teclea "Charizard" y presiona la tecla *Enter*.
- **Paso 4:** La aplicación valida el ingreso y muestra una alerta visual: *"¡Charizard ha sido añadido exitosamente! (Espacios ocupados: 1/6)"*.
- **Paso 5:** El usuario comete un error y decide presionar "4" (Deshacer última adición). La aplicación elimina a Charizard y muestra: *"Última acción revertida. El equipo vuelve a tener 0 integrantes."*

Funcionalidad: Analizar Debilidades

- **Paso 1:** Con un equipo ya formado, el usuario presiona la opción "3" (Analizar Debilidades) del submenú de Gestión.
- **Paso 2:** La aplicación muestra un reporte en pantalla indicando: *"Análisis de Vulnerabilidades: Tienes 2 Pokémon con debilidad extrema (x4) al tipo ELÉCTRICO"*.
- **Paso 3:** El sistema pausa la pantalla con el mensaje *"Presione cualquier tecla para volver al menú..."*, esperando la interacción del usuario.

Sección 2: Descripción de la Solución (Arquitectura e Ingeniería)

2.1 Entidades del Sistema (Tipos de Datos Propios)

- **Pokémon:** Entidad fundamental que modela a las criaturas. Sus atributos incluyen: **ID** (numérico entero), **Nombre** (cadena de caracteres), **Forma regional** (cadenas de caracteres), **Tipo1** y **Tipo2** (cadenas de caracteres), **Generación** (valores numéricos enteros), además de sus estadísticas base como **HP**, **Ataque**, **Defensa** y **Velocidad** (valores numéricos enteros).
- **Equipo:** Entidad gestora que agrupa lógicamente a las entidades Pokémon seleccionadas por el usuario, manteniendo un atributo contador para el límite máximo de integrantes.

2.2 Selección y Justificación de TDAs (Nivel Conceptual)

- **Mapa (Diccionario):** Se utilizará para gestionar el catálogo completo. *Justificación:* La operación más frecuente del usuario será buscar un Pokémon por su nombre. Un Mapa asocia directamente una "clave" (el nombre) a un "valor" (la entidad Pokémon), permitiendo un acceso instantáneo sin tener que revisar todo el registro secuencialmente.
- **Pila (Stack):** Se utilizará para administrar el Equipo activo. *Justificación:* Modela a la perfección la funcionalidad de "Deshacer". La lógica del equipo exige que el último Pokémon en ser ingresado sea obligatoriamente el primero en ser retirado (LIFO - *Last In, First Out*) si el usuario decide revertir su acción.
- **Grafo Dirigido:** Se utilizará para modelar el sistema de combate. *Justificación:* Los tipos elementales forman una red de relaciones donde un nodo (Fuego) tiene una relación de daño hacia otro (Planta) con un "peso" específico (ej. daño x2). El Grafo es el modelo matemático exacto para representar estas debilidades.

2.3 Estructuras de Datos Elegidas (Nivel de Implementación)

- **TDA Mapa -> Tabla Hash:** Se implementará usando una función de *hashing* para cadenas de texto, resolviendo las colisiones mediante Listas Enlazadas Simples (*Separate Chaining*). *Justificación:* Una Tabla Hash bien dimensionada permite búsquedas en tiempo promedio $O(1)$. Buscar en un arreglo lineal de más de 1000 Pokémon tomaría $O(n)$, lo cual es ineficiente computacionalmente.
- **TDA Pila -> Arreglo Estático:** Se programará mediante un arreglo de punteros de tamaño fijo con una variable índice tope. *Justificación:* Dado que el tamaño máximo del equipo es estrictamente 6 por reglas del juego, asignar memoria dinámica es innecesario. Un arreglo de tamaño 6 ofrece la máxima eficiencia temporal y espacial posible para las operaciones de la pila.

- **TDA Grafo -> Matriz de Adyacencia:** Se implementará como una matriz bidimensional estática de float de 18x18 (correspondiente a los 18 tipos elementales). *Justificación:* Conocer el multiplicador exacto de daño entre dos tipos toma un tiempo de consulta inmediato de $O(1)$, superando en velocidad de lectura a una Lista de Adyacencia.

2.4 Algoritmos y Complejidad por Funcionalidad

- **Búsqueda (Por Nombre):** El algoritmo calcula el valor *hash* del string ingresado, accede al índice del arreglo principal y recorre la lista enlazada en caso de colisión.
 - **Complejidad Temporal:** $O(1)$ en caso promedio (si la función *hash* distribuye bien), llegando a $O(n)$ en el peor de los casos teóricos.
 - **Complejidad Espacial:** $O(1)$ extra de memoria, ya que la consulta opera sobre los datos ya cargados.
- **Mecánica Deshacer (Pop en Pila):** El algoritmo simplemente decrementa en 1 la variable entera tope del arreglo estático que simula la Pila, ignorando el dato anterior.
 - **Complejidad Temporal:** $O(1)$, la operación es matemáticamente instantánea.
 - **Complejidad Espacial:** $O(1)$, no requiere asignación de memoria adicional.
- **Analizar Debilidades:** El algoritmo itera sobre los 'k' Pokémon actualmente en la Pila. Por cada uno, extrae sus tipos y consulta la Matriz de Adyacencia iterando sobre los 't' tipos posibles (18), acumulando los multiplicadores en un arreglo temporal de vulnerabilidades.
 - **Complejidad Temporal:** $O(1)$. Aunque la fórmula es $O(k * t)$, tanto el tamaño del equipo ($k \leq 6$) como los tipos ($t = 18$) son constantes pequeñas y estrictamente acotadas.
 - **Complejidad Espacial:** $O(1)$, ya que solo requiere un arreglo temporal estático de 18 posiciones para realizar la sumatoria de debilidades antes de imprimir.

Sección 3: Aspecto Desafiante

El proyecto "PokeHash" es un sistema complejo que exige un esfuerzo significativo debido a la alta integración técnica de múltiples Estructuras de Datos y la correcta gestión de memoria en C. El desafío principal radica en tres puntos:

1. **Parseo de datos no triviales:** Extraer y limpiar eficientemente la información de un archivo .csv con más de 1000 registros, gestionando correctamente los campos vacíos (Pokémon de tipo único) y convirtiendo las cadenas numéricas a tipos de datos enteros.

2. **Implementación desde cero:** El lenguaje C carece de colecciones nativas. El equipo debe programar manualmente una Tabla Hash robusta, diseñando una función matemática de dispersión eficiente para *strings* y administrando correctamente los punteros (*malloc/free*) para evitar fugas de memoria (*memory leaks*).
3. **Cruce de datos relacionales:** La funcionalidad de análisis táctico exige coordinar los datos extraídos de la Tabla Hash, mantenerlos activos en la Pila del equipo y cruzarlos algorítmicamente contra la Matriz de Adyacencia para calcular multiplicadores compuestos (ej. doble debilidad), sintetizando una abstracción matemática en un reporte claro por consola.

Sección 4: Distribución de roles y responsabilidades

Para facilitar el trabajo colaborativo y asegurar la calidad del desarrollo en C, el equipo ha definido la siguiente estructura de roles, donde cada integrante asume responsabilidades transversales vitales para el ciclo de vida del software:

1. Coordinador de Proyecto (Cristóbal)

Encargado de organizar y supervisar el ritmo de desarrollo global del equipo, asegurando la cohesión del trabajo.

- **Tareas clave:** Definir y mantener actualizado el cronograma de tareas semanales. Convocar y moderar las reuniones técnicas del equipo. Asegurar que todos los miembros comprendan la arquitectura propuesta y resolver cuellos de botella que afecten el avance del código.

2. Comunicador (Sebastián)

Responsable de la fluidez informativa interna y el enlace externo oficial con la cátedra.

- **Tareas clave:** Actuar como puente directo con el profesor y ayudantes para clarificar dudas de implementación. Organizar y redactar la documentación formal del proyecto (actas, requisitos, informes). Asegurar que el repositorio de código y documentos esté accesible y sincronizado para todo el equipo.

3. Gestor de Calidad (Simón)

Encargado de definir las metodologías ágiles de trabajo y resguardar la robustez técnica del software.

- **Tareas clave:** Supervisar el correcto uso del control de versiones (Git) evitando conflictos en las ramas (*branches*). Promover revisiones de código cruzadas (*code reviews*) para evitar malas prácticas de punteros. Organizar

retrospectivas semanales para optimizar el proceso de desarrollo y evitar la acumulación de requerimientos.

4. Responsable de Integración y Consistencia (Bruno)

Supervisor final de la arquitectura de *software*, garantizando que los módulos independientes funcionen como un sistema unificado.

- **Tareas clave:** Coordinar la unión de la lectura de archivos (CSV) con las estructuras de datos en memoria (Tabla Hash y Pila). Auditar que la nomenclatura de variables y *structs* sea consistente en todo el código fuente. Liderar las pruebas de estrés (ingreso de datos erróneos) para verificar que la aplicación no sufra *Segmentation Faults* y cumpla la rúbrica al pie de la letra.